

Project #3: Annotated Code Document

Client: Federal Highway Administration (FHWA)

Date: March 31, 2026

Code (Python)	Annotations
<code>import pandas as pd</code>	Imports the python library used for data manipulation.
<code>import numpy as np</code>	Imports the python library used for advanced math and handling large numbers.
<code>import matplotlib.pyplot as plt</code>	Imports the library used for plotting data to generate graphs and visualizations.
<code>import seaborn as sns</code>	Imports a python library that makes plots display more neatly and professionally.
<code>sns.set_style('whitegrid')</code>	Sets a visual theme with a grid, making it easier to read exact values.
<code>nhts_data = pd.read_csv('NHTS(in).csv')</code>	Commands Python to read the survey data file into a table.
<code>ngsim_data = pd.read_csv('NGSIM(in).csv')</code>	Reads the driving behavior data file into a table.
<code>plt.figure(figsize=(12, 6))</code>	Defines the dimensions of the graph window (12" x 6").
<code>sns.countplot(..., palette='inferno')</code>	Counts the occurrences of each vehicle type and colors them using the "Inferno" theme from the seaborn library.
<code>plt.hist(..., bins=20, color='skyblue')</code>	Groups "Vehicle Age" into 20 different ranges to show frequency.
<code>nhts_data.boxplot(column='...', by='...')</code>	Creates a box-and-whisker plot to show the median and spread of the data.
<code>plt.xticks(rotation=45)</code>	Rotates the X-axis text by 45 degrees so long labels don't overlap.
<code>plt.tight_layout()</code>	Automatically adjusts margins so no titles or labels are cut off.
<code>trajectory_number = 2</code>	Defines which specific vehicle interaction from the thousands in the data we want to see.
<code>data_subset = ngsim_data[...]</code>	Filters the master list down to only the data points for our chosen vehicle pair.
<code>time_data = data_subset['Time'].values</code>	Saves the "Time" column as an array for the x axis of our graphs.
<code>gap_dist = leader_pos - follower_pos</code>	Calculates the instantaneous distance between the front and rear vehicle.
<code>plt.plot(..., label='Leader Speed')</code>	Draws a line representing the lead car's speed and labels it for the legend.

<code>plt.legend()</code>	Adds a key (legend) to the plot so the viewer knows which line is the Leader vs. Follower.
<code>def idm_acceleration(...):</code>	Defines a custom math function that executes the IDM formula.
<code>s_star = s0 + v * T + ...</code>	Calculates the "Desired Minimum Gap" based on velocity and safe time headway.
<code>max(s_star, s0)</code>	Ensures the car never plans for a gap smaller than the stopped distance.
<code>acceleration = a * (1 - ...)</code>	The final IDM equation calculating the predicted acceleration.
<code>np.zeros(n_steps)</code>	Makes room for empty space in memory to store our 800+ simulation results.
<code>for i in range(n_steps - 1):</code>	Starts a loop that calculates for every 0.1-1 second interval.
<code>current_s = max(leader - follower, 0.1)</code>	Calculates current gap; the 0.1 prevents "divide by zero" errors if a crash occurs.
<code>sim_speed[i+1] = sim_speed[i] + acc * dt</code>	Updates velocity by adding (Acceleration $\times$ Time).
<code>plt.tight_layout()</code>	Automatically adjusts the padding so that labels and titles aren't cut off when saving the image.
<code>plt.plot(..., alpha=0.5)</code>	Sets the line transparency to 50% so overlapping lines are still visible.
<code>plt.plot(..., color='red', linewidth=2)</code>	Highlights the simulation line in bold red to distinguish it from the raw data.